

Beyond Automation: Verifiable AI, Human Oversight, and Trustworthy Supply Chain Defense

The integration of Artificial Intelligence (AI) into software supply chains offers scalable defense but introduces critical trust deficits. Current machine learning models often generate untrustworthy alerts by relying on irrelevant syntactic features rather than semantic logic, leading to high false positive rates and developer fatigue [229] [295]. Furthermore, "vibe coding" practices have exposed new vectors for compromise, such as artifact hygiene failures, necessitating specialized pre-publish gates like VibeGuard to detect configuration drift missed by traditional scanners [24]. To mitigate these risks, a paradigm shift toward human-in-the-loop verification is essential. While agentic frameworks like CHASE and VulnAgent-X automate complex reasoning and evidence fusion, they cannot replace human responsibility in defining security policies and validating architectural integrity, particularly in safety-critical domains like autonomous vehicles [5] [42] [292]. Future security architectures must adopt Human-Certified Module Repositories to blend curated trust with automated analysis, ensuring AI serves as a rigorous assistant rather than an unchecked authority [77] [Software Security Analysis in 2030 and Beyond: A Research Roadmap]. Ultimately, robust supply chain security requires synthesizing explainable AI, neuro-symbolic reasoning for heterogeneous artifacts, and continuous human oversight to counter evolving threats like malicious skills and model poisoning [MalSkills: Detecting Malicious Skills via Neuro-Symbolic Reasoning across Heterogeneous Artifacts] [296].

Introduction to the Evolving AI Software Supply Chain Landscape

The modern software supply chain has undergone a radical transformation, evolving from a linear development pipeline into a complex, interdependent ecosystem where the reliance on open-source components and AI-generated code introduces profound security risks. This shift has expanded the attack surface significantly, with open-source software now constituting over 90% of the codebase in many commercial applications, thereby creating numerous opportunities for malicious actors to inject compromised code into downstream systems [235]. The ubiquity of package managers in ecosystems such as npm, PyPI, and RubyGems has facilitated rapid development but simultaneously enabled the proliferation of supply chain attacks, ranging from dependency confusion to the injection of malicious packages that can affect thousands of dependent projects [Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks]. Recent data indicates an exponential increase in these incidents, with attacks targeting build infrastructure and trusted development procedures becoming increasingly sophisticated, rendering conventional post-build integrity mechanisms insufficient [4].

A critical dimension of this evolving landscape is the emergence of the Large Language Model (LLM) supply chain, which introduces unique structural vulnerabilities distinct from traditional software dependencies. Empirical studies reveal that the LLM supply chain exhibits a "locally dense, globally sparse" topology, where a few high-degree hub nodes, such as the transformers library, create single points of failure that can propagate critical vulnerabilities to hundreds of downstream packages [21]. The risk is compounded by the fact that vulnerabilities in the LLM ecosystem often remain undisclosed for significantly longer periods compared to general software ecosystems, with half of such vulnerabilities remaining unaddressed for over four years [16]. Furthermore, the integration of AI models into critical systems has created new attack vectors where adversaries target not just the code, but the data pipelines, model weights, and configuration files, as seen in recent exploits involving malicious configurations on platforms like Hugging Face [15]. These AI-specific threats are often exacerbated by the black-box nature of model components, leading to a landscape where many security issues related to models and data lack concrete, established solutions [293].

Traditional static analysis and rule-based scanners are increasingly inadequate against these sophisticated, evolving threats, particularly when dealing with opaque industrial binaries or the semantic nuances of AI-generated code. Conventional Software Composition Analysis (SCA) tools often suffer from high false-positive rates and fail to account for the semantic context of vulnerabilities, leading to alert fatigue and missed detections [86]. In response, the literature suggests a paradigm shift toward Artificial Intelligence, Large Language Models, and neuro-symbolic frameworks to enhance detection capabilities. For instance, neuro-symbolic approaches have been proposed to bridge the semantic gap in binary analysis by combining abstract interpretation with LLM-based reasoning, effectively suppressing hallucinations while capturing long-range vulnerability propagation [9]. Similarly, multi-agent systems leveraging LLMs have demonstrated superior performance in detecting malicious skills and packages by reasoning across heterogeneous artifacts, outperforming state-of-the-art baselines by significant margins ["Elementary, My Dear Watson." Detecting Malicious Skills via Neuro-Symbolic Reasoning across Heterogeneous Artifacts].

Despite the promise of AI-driven defenses, challenges remain regarding the reliability and adaptability of these systems. While LLMs show potential in replacing conventional scanners for identifying code errors and deprecated functions, they face limitations in memory complexity and handling unfamiliar data patterns [11]. To address these limitations, researchers are developing agentic AI frameworks that combine LLM reasoning with reinforcement learning and multi-agent coordination to create proactive, self-defending supply chains capable of adaptive mitigation [4]. These systems aim to move beyond mere provenance tracking, as provided by frameworks like SLSA and SBOMs, toward active vulnerability removal and real-time defense [179]. However, the effectiveness of SBOMs themselves is currently under scrutiny, with studies showing that inaccuracies in SBOM generation can lead to staggering false-positive rates in vulnerability scanning, necessitating more robust, lock-file-based generation methods enriched with function call analysis [127].

The stakes of these vulnerabilities are particularly high in specialized domains such as autonomous vehicles and blockchain, where software flaws can translate directly into physical safety hazards or massive financial losses. In the autonomous vehicle sector, the reliance on open-source stacks for perception and control means that compromised dependencies can cascade into unsafe maneuvers, highlighting the urgent need for security best practices embedded early in the development lifecycle [5]. Similarly, in the Web3 ecosystem, the immutable nature of smart contracts

intersects with traditional supply chain problems, creating a high-stakes environment where vulnerability detection tools must evolve to handle the unique constraints of blockchain technology [126]. As the industry grapples with these challenges, there is a growing consensus on the need for human-certified module repositories and governance frameworks that blend human oversight with automated analysis to ensure the trustworthiness of AI-assembled systems [77]. The convergence of these threats and the limitations of current defenses underscore the necessity for a holistic, defense-oriented approach that integrates semantic analysis, agentic autonomy, and rigorous provenance tracking to secure the future of the software supply chain [134].

The Expansion of Attack Surfaces and Threat Vectors

The modern software supply chain has undergone a radical transformation, evolving from a linear dependency model into a complex, multi-layered ecosystem where the attack surface extends far beyond traditional code injection. Historically, supply chain security focused primarily on dependency confusion and direct code tampering; however, contemporary threats now exploit the intricate trust relationships inherent in open-source ecosystems, build pipelines, and increasingly, the artificial intelligence components that underpin modern development. This expansion is characterized by sophisticated social engineering campaigns, the weaponization of build configurations, and novel AI-specific vectors such as model weight poisoning and malicious configuration files. The sheer scale of open-source software adoption, estimated to constitute over 90% of modern applications, provides a fertile ground for these attacks, as demonstrated by comprehensive taxonomies that map hundreds of unique attack vectors across the entire software lifecycle [135].

A critical vector in this expanded landscape is the manipulation of the human element through social engineering, which has become a primary method for gaining initial access to repositories. Attackers no longer rely solely on technical exploits but instead infiltrate development teams by exploiting social dynamics and team behaviors to become trusted contributors. Research indicates that signatures of team dynamics can be leveraged to predict susceptibility to such attacks, where adversaries manipulate interactions to inject malicious code or vulnerabilities [Leveraging Team Dynamics to Predict Open-Source Software Projects' Susceptibility to Social Engineering Attacks]. This approach was starkly illustrated in the XZ Utils supply chain attack, where attackers spent years cultivating trust within the community, eventually manipulating the project's infrastructure and CI/CD configurations to inject a backdoor into a fundamental Linux compression library [112]. Such incidents reveal that software engineering practices themselves can be weaponized, bypassing traditional security measures that focus exclusively on code logic rather than the socio-technical processes of development [255].

Parallel to social engineering, the build pipeline has emerged as a high-value target, with attackers shifting focus from source code to the automation scripts that compile and package software. Build systems, often written in scripting languages distinct from the core application, are frequently overlooked by static analysis tools designed for program code. The "Build Code is Still Code" paradigm highlights how poisoned build systems can circumvent detection by disabling security checks or injecting malicious artifacts during compilation, a tactic central to recent high-profile compromises [106]. Empirical studies confirm that nearly a quarter of supply chain attacks specifically exploit build vulnerabilities, utilizing vectors such as build script injection and dependency resolution abuse [167]. Furthermore, CI/CD pipelines themselves present a vast attack surface, where misconfigured scripts and outdated runtimes allow attackers to hide malicious code or exploit single points of failure in core scripts [100].

The integration of Artificial Intelligence into software development has introduced a new class of vulnerabilities that traditional detection mechanisms fail to address. The phenomenon of "vibe coding," where developers delegate code generation to AI assistants with minimal review, has created blind spots in security postures. The leak of Anthropic's Claude Code source map, attributed to a misconfigured packaging rule in an AI-generated project, underscores the gap between vulnerabilities introduced by AI and those current tooling is designed to find [24]. Beyond code generation, the AI supply chain faces unique threats through model hubs like Hugging Face, where pre-trained models and their associated configuration files can be weaponized. Malicious configuration files, originally intended for model setup, can execute unauthorized code, while unsafe serialization methods in model weights allow for remote code execution upon loading [15]. Large-scale instrumentation studies have further demonstrated that models using unsafe serialization methods can be exploited and shared, creating an unsafe environment for ML developers [59].

Attackers are also exploiting the semantic limitations of current scanners by abusing deep learning APIs to hide malicious functionalities such as file manipulation and network exfiltration within seemingly benign models [60]. The risk is compounded by the "hallucinating AI hijacking" attack, where large language models, when presented with specific contexts, may recommend malicious code or API endpoints that violate safety policies, effectively amplifying the attack surface through trojan-hosting repositories [275]. Additionally, the reliance on third-party dependencies in LLM projects introduces significant risks, with studies showing that a vast majority of LLMs include vulnerable dependencies in their configuration files, often remaining undisclosed for extended periods [16].

The convergence of these vectors-social engineering, build pipeline poisoning, and AI-specific exploits-creates a defense-in-depth challenge that requires moving beyond simple dependency scanning. Traditional tools often struggle with high false-positive rates or fail to detect stealthy attacks that leverage language-specific features or obfuscation techniques [119]. Consequently, the security community is shifting towards more holistic approaches, including the use of Large Language Models for semantic analysis of configurations and the development of frameworks that assess the trustworthiness of modules through human-certified repositories [77]. As the attack surface continues to expand, the distinction between accidental vulnerabilities and deliberate malicious injection becomes increasingly blurred, necessitating advanced detection strategies that account for the behavioral intent of code and the provenance of AI-generated artifacts [133].

The Dual Role of AI in Defense and Risk

The integration of Artificial Intelligence into the software supply chain presents a profound paradox: AI serves as both a sophisticated shield against vulnerabilities and a novel vector for systemic compromise. On the defensive front, deep learning architectures have evolved beyond simple pattern matching to capture the semantic and structural nuances of source code. Graph Neural Networks (GNNs), in particular, have demonstrated superior capabilities by representing code as graphs, allowing models to learn complex relationships between nodes and edges that traditional static analyzers and linear sequence encodings often miss [274]. This graph-based paradigm enables the detection of vulnerabilities based on intrinsic logic rather than superficial syntax. Furthermore, the granularity of detection has improved significantly; frameworks like LineVD leverage GNNs to pinpoint specific vulnerable statements rather than merely flagging entire functions, thereby enhancing the interpretability and utility of automated audits [221]. Efficiency has also been addressed, with lightweight models like VulGNN achieving performance parity with Large Language Models (LLMs) while being substantially smaller and faster to retrain, making them viable for real-world deployment in development pipelines [294].

Despite these advancements, the efficacy of AI-driven defense is frequently undermined by challenges related to data quality, model generalization, and adversarial robustness. A recurring theme in the literature is the significant performance degradation of deep learning models when transitioning from curated academic benchmarks to real-world, out-of-distribution codebases [283]. Studies reveal that many state-of-the-art models overfit to unrelated artifacts, such as variable names or specific dataset distributions, rather than learning the intrinsic logic of vulnerabilities, leading to a sharp drop in accuracy when evaluated on time-wise split datasets or industrial code [295]. The issue of data imbalance, where vulnerable code samples are vastly outnumbered by secure code, further complicates model training, often resulting in high false-negative rates unless specific loss functions or sampling strategies are employed [An Empirical Study of the Imbalance Issue in Software Vulnerability Detection]. Additionally, the "curse of variables"-the infinite possibilities for naming identifiers-can hinder a model's ability to generalize functionality across different code expressions, necessitating abstract syntax graphs that normalize variable names to capture true semantic intent [107].

Conversely, the AI ecosystem itself has become a fertile ground for supply chain attacks, introducing risks that traditional security tools are ill-equipped to handle. The rapid proliferation of pre-trained models and libraries on platforms like Hugging Face and PyPI has expanded the attack surface to include malicious model configurations, unsafe serialization methods, and poisoned dependencies. Empirical investigations highlight that security challenges related to AI models and data often lack concrete solutions, with many vulnerabilities stemming from the black-box nature of AI components and their complex dependencies [293]. Specific threats include the exploitation of configuration files to execute unauthorized code and the use of unsafe serialization protocols like Python's pickle, which can be manipulated to trigger remote code execution upon model loading [15] [267]. The structural characteristics of the LLM supply chain, described as "locally dense, globally sparse," mean that vulnerabilities in critical hub nodes can cascade to hundreds of downstream packages, amplifying the impact of a single compromise [21].

Moreover, the generative capabilities of LLMs introduce a unique class of risks where the tools intended to secure code may inadvertently recommend compromised APIs or trojan-hosting repositories. Research demonstrates that foundational models, while guarded against overtly harmful prompts, can be coerced through context-shifting attacks to suggest malicious code or vulnerable dependencies, effectively hijacking the development process [275]. This risk is compounded by the phenomenon of "vibe coding," where developers delegate code generation to AI assistants with minimal review, creating blind spots for artifact hygiene and packaging-configuration drift that existing static analysis tools fail to cover [24]. Consequently, while AI offers scalable solutions for vulnerability detection, its integration necessitates a rigorous re-evaluation of trust mechanisms. The dual nature of AI demands a holistic security strategy that addresses not only the detection of traditional software flaws but also the mitigation of risks inherent to the AI development lifecycle, potentially through agentic systems capable of autonomous reasoning and mitigation [4] [42].

Scope and Domain Diversity

The vulnerability landscape within the AI software supply chain is profoundly heterogeneous, shifting from the high-volume, low-latency risks of general-purpose registries to the safety-critical, immutable constraints of specialized domains. In general-purpose ecosystems such as npm and PyPI, the primary threat surface is defined by the sheer scale of interdependence and the openness of the publication process. Research indicates that these registries exhibit a "locally dense, globally sparse" topology, where a vast majority of dependency trees are small, yet a few critical hub nodes dominate the ecosystem, acting as amplifiers for cascading risks [21]. In these environments, attackers frequently exploit human error through typosquatting and combosquatting, or compromise maintainer accounts to inject malware into widely used libraries [1] [157]. The density of these networks means a single vulnerable dependency can expose thousands of downstream projects, creating a propagation dynamic where critical vulnerabilities reach an average of 142 nodes at the second layer of dependency trees [21]. Consequently, vulnerability management in this domain relies heavily on automated detection of suspicious metadata and install scripts, as well as rapid response mechanisms to dependency confusion attacks [42].

In stark contrast, the domain of autonomous vehicles (AVs) presents a fundamentally different set of challenges where software vulnerabilities translate directly into physical safety hazards. Unlike general software, AVs are tightly coupled cyber-physical systems where flaws in the software supply chain can manifest as unsafe maneuvers or catastrophic accidents [5]. The architecture of AVs relies on complex stacks involving perception, localization, and planning modules, often built upon open-source foundations like Autoware and Apollo. Here, the scope of vulnerability extends beyond traditional code defects to include sensor fusion errors, real-time decision-making failures, and the integrity of third-party datasets [5]. The lack of reproducible builds and the difficulty in performing binary analysis on embedded components further complicate the security posture, requiring domain-specific adaptations that account for the strict latency and reliability constraints of automotive systems [5]. This domain necessitates a shift from purely code-centric analysis to system-level risk scoring that reflects both technical severity and safety impact, as traditional static analyzers often fail to capture the nuances of safety-critical failure modes [5].

Similarly, the Ethereum blockchain and smart contract domain introduces unique risks centered on financial loss and the immutability of deployed code. Vulnerabilities in smart contracts, such as reentrancy attacks, can lead to irreversible theft of assets, as evidenced by historical incidents like the DAO hack [141]. The scope here is defined by the need for precise static and dynamic analysis of bytecode and Solidity source code, where traditional software patching mechanisms are often unavailable post-deployment. Machine learning approaches in this domain must contend with restricted vulnerability coverage and dataset construction flaws, highlighting a gap between academic research and the practical necessities of securing decentralized finance applications [141]. Furthermore, research software and industrial control systems operate in environments characterized by opaque binaries and a lack of source-level transparency. In these sectors, the "semantic gap" prevents conventional Software Composition Analysis tools from effectively identifying vulnerabilities, necessitating neuro-symbolic frameworks that can reconstruct behavioral semantics from stripped binaries [9].

The divergence in vulnerability characteristics across these domains also influences the applicability of automated vulnerability detection (AVD) techniques. Research indicates that current AVD methods often suffer from an "applicability gap," where models trained on specific tasks or languages fail to generalize to the diverse realities of industrial or safety-critical software [162]. For instance, deep learning models that perform well on general source code may struggle with the specific semantic structures of smart contracts or the binary constraints of embedded automotive systems [158]. This fragmentation suggests that effective governance and AI adaptation must be domain-specific, leveraging tailored metrics such as the Vulnerability Propagation Scoring System (VPSS) for supply chains or specialized agent architectures for repository-level analysis [113] [292]. Additionally, emerging threats such as "evil configurations" in AI model repositories and social engineering attacks targeting development teams further diversify the threat landscape, requiring defenses that extend beyond code analysis to include configuration validation and team dynamic assessment [15] [Leveraging Team Dynamics to Predict Open-source Software Projects' Susceptibility to Social Engineering Attacks]. Ultimately, recognizing this domain diversity is essential for developing robust security strategies that address the distinct threat surfaces of each sector, from the rapid churn of open-source packages to the rigid safety requirements of autonomous systems.

Taxonomy and Characterization of Supply Chain Threats

Understanding the specific mechanisms of supply chain attacks is a prerequisite for developing effective defenses, requiring a shift from broad categorizations to detailed taxonomies that account for entry points, execution triggers, and malicious intent. Recent literature distinguishes sharply between vulnerable packages, which contain accidental design flaws or negligence, and malicious packages, where code errors are implemented selectively with the intention to cause damage or steal information [Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks]. This distinction is critical because malicious code often employs obfuscation and evasion techniques that differ fundamentally from the patterns found in accidental vulnerabilities, necessitating specialized detection strategies. Foundational work in this domain has moved toward technology-independent classifications to address the ubiquity of open-source software, which may constitute more than 90% of commercial application code [235]. One such comprehensive effort proposes a general taxonomy covering 117 unique attack vectors derived from an examination of 370 resources, highlighting the vast attack surface where malicious code can be injected into components and infect downstream users [235]. Similarly, other researchers have presented an attack tree covering 107 unique vectors linked to 94 real-world incidents, validated by surveys of domain experts to ensure utility across various use cases [135].

The mechanisms of injection vary significantly across the software lifecycle, with attackers leveraging specific package manager capabilities to achieve arbitrary code execution. Research identifies specific install-time and runtime techniques across seven ecosystems, demonstrating how trust models are bypassed through vectors such as typosquatting and dependency confusion [The Hitchhiker's Guide to Malicious Third-Party Dependencies]. Typosquatting exploits user error by registering packages with names similar to legitimate ones, while dependency confusion attacks exploit the resolution logic of package managers by uploading malicious packages with names identical to internal private dependencies [74]. The effectiveness of these attacks is often facilitated by metadata signals, such as expired email domains and inactive maintainers, which allow attackers to hijack accounts and push malicious updates [157]. Beyond name-based attacks, more subtle vectors have emerged, such as class hijacking in Java, where a transitive dependency shadows a legitimate class to alter behavior deep within the dependency tree [206].

The complexity of modern dependency networks further exacerbates these threats, creating structures described as "locally dense, globally sparse," where a few high-degree hub nodes dominate the ecosystem [21]. In such topologies, vulnerabilities or malicious code in critical hubs can cascade to hundreds of downstream packages, creating a disproportionate risk that challenges conventional defense strategies. For instance, local dependency versioning strategies like pinning, often advocated to prevent malicious updates, may inadvertently increase exposure in larger dependency graphs due to the specifics of resolution mechanisms [70]. This cascading effect is not limited to public registries but extends to the build infrastructure itself, with empirical taxonomies revealing that nearly 24% of supply chain attacks exploit build vulnerabilities, particularly through dependency confusion and build script injection [167].

Emerging research also highlights the critical role of social engineering and the manipulation of community processes in facilitating technical compromises. Sophisticated attacks, such as the incident involving XZ Utils, demonstrate how adversaries can manipulate software engineering practices, including community management and CI/CD configurations, to establish legitimacy and inject backdoors into fundamental libraries [112]. This aligns with findings that systematically explore tactics used to trick software engineers into delivering malicious software during critical development lifecycle steps, indicating that human factors are as vulnerable as technical pipelines [255]. Furthermore, the rise of AI introduces new vectors, with malicious configuration files in model repositories identified as an overlooked threat capable of executing unauthorized code [15]. To address the fragmentation in understanding these diverse threats, recent work argues for a defense-oriented approach that focuses on holistic solutions and causal relationships between supply chain elements [134]. Additionally, extending logical attack graph formalisms enables the modeling of complex, multi-stage attacks that combine network vulnerabilities with supply chain compromises, moving the field beyond static lists of vulnerabilities toward dynamic models that account for intent, ecosystem topology, and the interplay between human and automated processes [124].

Registry-Specific and Ecosystem Attack Vectors

Public package registries serve as the foundational infrastructure for modern software development, yet their open nature and reliance on automated dependency resolution make them primary targets for supply chain attacks. The threat landscape is characterized by a diverse array of attack vectors that exploit human error, lax verification processes, and ecosystem-specific architectural nuances. Common attacks include typosquatting, where attackers publish packages with names syntactically similar to legitimate ones, and combosquatting, which exploits confusion regarding the order of words in package names [1]. These techniques rely heavily on developer mistakes during manual installation, allowing malicious code to infiltrate projects under the guise of trusted libraries [The Hitchhiker's Guide to Malicious Third-Party Dependencies]. Beyond simple naming confusion, attackers leverage more sophisticated methods such as dependency confusion, where a malicious package with a name matching an internal private package is uploaded to a public registry, tricking build systems into downloading the external, compromised version [74].

While the fundamental goals of these attacks remain consistent across ecosystems, the specific risk profiles and exploitation mechanisms vary significantly between Node.js, Python, and Java environments. The npm ecosystem, characterized by its massive volume and granular package structure, faces unique challenges related to behavioral ambiguity and the prevalence of trivial packages. Research indicates that nearly 18% of npm packages are "trivial," containing minimal code yet introducing deep dependency chains that amplify vulnerability exposure [234]. The sheer scale of npm complicates detection, as malicious actors can hide malicious logic within complex installation scripts or obfuscated code that static analysis tools often miss [215]. Furthermore, the npm dependency resolution mechanism introduces specific risks; for instance, pinning dependencies, often recommended as a security best practice, can paradoxically increase the attack surface for malicious updates in large dependency graphs due to how npm resolves version conflicts [70]. The ecosystem also suffers from weak links such as maintainer accounts associated with expired email domains, which allow attackers to hijack thousands of packages by reclaiming domain ownership [157].

In contrast, the Python ecosystem (PyPI) is frequently targeted due to the ease of exploiting the pip package manager and the lack of rigorous pre-publication scanning. Attacks in this environment often involve malicious commits or the injection of code during the installation phase, with studies showing that over 74% of malicious packages successfully enter end-user projects through source code installation [146]. The Python ecosystem also faces significant challenges regarding the persistence of malicious packages on mirror servers, with over 72% remaining available long after discovery [146]. Additionally, the rise of AI and machine learning has introduced new vectors within PyPI, particularly through unsafe serialization methods in model repositories like Hugging Face, which can lead to arbitrary code execution upon loading [59]. Detection in PyPI is further complicated by the fact that many malicious packages exhibit behavior sequences that are difficult to distinguish from benign activity without sequential modeling [74].

The Java ecosystem, primarily centered around Maven, presents a distinct set of risks driven by its deep dependency trees and classloading mechanisms. A notable attack vector specific to Java is "class hijacking," where an attacker crafts a class that shadows a legitimate class deep within the dependency tree, effectively altering the behavior of direct dependencies [206]. This type of attack exploits the way Maven resolves dependencies and how the Java Virtual Machine loads classes, posing a severe risk to applications that rely on transitive dependencies they do not directly control. Moreover, vulnerability propagation in Maven is exacerbated by the slow adoption of fixes; research shows that vulnerabilities in Java packages can affect a median of 59 releases, significantly higher than in other ecosystems [211]. The complexity of Java build processes also introduces risks related to build compromises, where attackers inject malicious code during the compilation phase, a vector that is particularly difficult to detect without reproducible builds [150].

Despite these ecosystem-specific differences, there is a growing consensus that malicious behaviors share commonalities that can be leveraged for cross-ecosystem detection. Recent advancements propose using language-independent features and behavior sequence modeling to detect malicious packages in both npm and PyPI simultaneously, overcoming the limitations of siloed detection tools [74] [108]. However, existing defenses often struggle with high false positive rates or fail to account for the dynamic nature of attacks, such as runtime code injection or the use of living-off-the-land techniques [119]. The diversity of attack vectors necessitates a shift from generic security measures to tailored strategies that address the specific resolution logic, package metadata structures, and execution models of each registry. For instance, while metadata analysis is effective for detecting typosquatting in npm,

it may be less effective against sophisticated class hijacking in Java without deep static or dynamic analysis [56] [182]. Ultimately, securing these ecosystems requires a multi-layered approach that combines registry-level scanning, improved dependency resolution policies, and runtime enforcement mechanisms tailored to the unique architectural constraints of each environment.

AI Model and Configuration Vulnerabilities

The integration of artificial intelligence into the software supply chain has introduced a distinct class of vulnerabilities that extend beyond traditional code dependencies to encompass model artifacts, configuration files, and the generative processes themselves. While conventional supply chain security focuses on package managers and source code repositories, the AI supply chain relies heavily on model hubs like Hugging Face, where the boundary between data, configuration, and executable code is often blurred. Research indicates that configuration files, originally designed to specify model parameters and initialization settings, have become a primary vector for unauthorized code execution. A comprehensive study of the Hugging Face ecosystem reveals that malicious actors can exploit these configurations to perform file operations, website interactions, and repository manipulations, effectively bypassing security measures that focus solely on model weights [15]. This vulnerability is exacerbated by the reliance on unsafe serialization methods, such as Python's pickle, which allows for object injection attacks. Investigations into large-scale exploit instrumentation demonstrate that a significant portion of models on Hugging Face utilize these unsafe serialization protocols, creating an environment where loading a model can inadvertently trigger remote code execution (RCE) or data exfiltration [59].

The threat landscape is further complicated by the abuse of deep learning APIs, which attackers can leverage to embed malicious functionalities within seemingly benign models. Unlike traditional malware that hides in binary blobs, AI-specific attacks can abuse hidden functionalities in frameworks like TensorFlow to read files, modify system states, or establish network connections during the model's prediction phase. Existing scanning tools often fail to detect these threats because they rely on syntactic analysis rather than semantic understanding of API usage, allowing stealthy abuses of DL APIs to persist undetected in popular model hubs [60]. Furthermore, the persistence of pickle-based vulnerabilities remains a critical issue; systematic disclosures reveal that current scanners miss numerous loading paths and exploitable gadgets, enabling attackers to employ techniques like Exception-Oriented Programming (EOP) to bypass detection mechanisms with high success rates [267]. The scale of this issue is underscored by end-to-end pipelines like MalHug, which, when deployed on mirrored instances of model hubs, have uncovered hundreds of malicious models and dataset loading scripts capable of reverse shells and credential theft [244].

Parallel to the risks inherent in model repositories, the rise of AI-assisted code generation, often termed "vibe coding," has introduced new vectors for supply chain compromise through misconfigured packaging and accidental exposure. The reliance on AI to generate code without rigorous manual review has led to incidents where proprietary source maps and sensitive logic are inadvertently exposed due to packaging rule errors, as seen in the Claude Code source leak. This specific failure mode highlights a gap in existing static analysis and secret-scanning tools, which are not designed to catch configuration drifts or artifact hygiene issues introduced by AI agents. To address this, frameworks like VibeGuard have been proposed to act as pre-publish security gates, targeting blind spots such as source-map exposure and packaging-configuration drift that traditional tools overlook [24]. Additionally, the dependency supply chain for Large Language Models (LLMs) itself is fraught with risk; empirical studies show that a vast majority of LLMs include vulnerable dependencies in their configuration files, with vulnerabilities remaining undisclosed for significantly longer periods compared to the broader Python ecosystem [16]. The structural characteristics of the LLM supply chain, described as "locally dense, globally sparse," mean that critical vulnerabilities in hub nodes like transformers can cascade to affect thousands of downstream packages, amplifying the impact of any single compromise [21].

The intersection of AI generation and supply chain integrity also presents unique challenges regarding "protestware" and intentional disruption by maintainers, as well as the hallucination of malicious code recommendations by LLMs. Foundational models, while guarded against overtly harmful prompts, may drop their safety filters when presented with context-shifted programming challenges, potentially recommending API endpoints or code structures that facilitate "living off the land" attacks or domain squatting [275]. This aligns with broader findings that AI systems rest on software with low integrity mechanisms, lacking verifiability, versioning, and traceability across the data acquisition to inference lifecycle [29]. The complexity is such that even the build systems supporting AI projects are vulnerable; poisoned build scripts can circumvent traditional code checkers, necessitating development phase isolation strategies to secure the pipeline [106]. Moreover, the reliance on open-source components in AI projects means that social engineering attacks targeting development teams can lead to the injection of malicious code, a risk that is difficult to mitigate when the code itself is partially generated or assembled by AI agents [Leveraging Team Dynamics to Predict Open-source Software Projects' Susceptibility to Social Engineering Attacks].

Addressing these multifaceted vulnerabilities requires a shift from reactive scanning to proactive, semantic-aware defense mechanisms. While traditional Software Composition Analysis (SCA) tools struggle with the dynamic nature of AI artifacts, new approaches leverage Large Language Models to analyze configuration files in the context of runtime code, achieving higher accuracy in detecting suspicious elements [15]. Similarly, neuro-symbolic frameworks are being developed to reconstruct behavioral semantics from opaque binaries and models, combining abstract interpretation with LLM prompting to suppress hallucinations and identify zero-day attack patterns [9]. The emergence of agentic AI for autonomous defense suggests a future where multi-agent systems coordinated by LLMs can actively identify and mitigate vulnerabilities within CI/CD pipelines, moving beyond mere provenance tracking to active vulnerability removal [4]. However, the effectiveness of these solutions depends on the availability of high-fidelity data and the ability to distinguish between benign AI-generated anomalies and genuine malicious intent, a challenge that remains central to securing the evolving AI supply chain.

Cascading, Transitive, and Silent Risks

The propagation of vulnerabilities through software supply chains represents a critical challenge, primarily driven by the complex web of transitive dependencies that characterize modern development ecosystems. Vulnerabilities rarely remain isolated within direct dependencies; instead, they cascade through deep dependency trees, often compromising applications that have no direct relationship with the flawed component. This phenomenon is particularly acute in hub-and-spoke architectures where a single popular package serves as a foundational node for thousands of downstream projects. Research indicates that transitive dependencies often outnumber direct dependencies significantly, creating a vast attack surface where a flaw in a deep dependency can infect hundreds of thousands of packages [14]. In the npm ecosystem alone, studies have identified over 200,000 packages infected through their dependencies, highlighting the scale of this cascading risk [14]. The depth of these risks is substantial, with empirical evidence suggesting that vulnerabilities predominantly emerge at a dependency depth of three levels or higher, indicating that direct and secondary dependencies often exhibit lower vulnerability density compared to their transitive counterparts [39].

The mechanics of this propagation are further complicated by the specific resolution rules of package managers, which can lead to "dependency hell" or the inadvertent inclusion of vulnerable versions if not managed with precise graph-based tracking. Traditional reachability analysis often fails to account for these ecosystem-specific resolution rules, resulting in inaccurate assessments of vulnerability scope [122]. To address this, advanced frameworks utilizing hierarchical worklist-based algorithms and knowledge graphs have been proposed to map whole-ecosystem vulnerability propagation, offering a more granular view of how threats evolve across dependency trees [113]. Furthermore, the structural topology of these ecosystems, such as the "locally dense, globally sparse" nature observed in Large Language Model supply chains, means that cascading risks are often concentrated in critical hub nodes, where a single compromise can affect over a thousand downstream packages [21].

Compounding the risk of cascading failures is the phenomenon of "silent" vulnerabilities, where maintainers patch security issues without clear communication, version bumps, or public advisories. This lack of transparency leaves downstream users unaware that they are running compromised code, even when a fix exists. Studies reveal that a significant proportion of dependency vulnerabilities have fixes available within the same major release range, yet projects remain exposed because developers are not notified of the silent updates [211]. In fact, nearly 70% of security vulnerabilities in open-source software are patched before being reported in public databases, creating a window of exposure where users rely on outdated information [177]. To mitigate this, researchers have developed explainable AI models capable of predicting silent dependency alerts by analyzing commit messages and code changes, providing users with key aspects such as root causes and attack vectors to enhance trust and detection speed [10].

Beyond silent patches, the supply chain faces the threat of dormant malicious code and high-stealth backdoors that evade traditional static analysis until triggered by specific conditions. These attacks often leverage the trust placed in long-term contributors or manipulate software engineering practices to inject malicious code that remains inactive during standard testing phases [Wolves in the Repository: A Software Engineering Analysis of the XZ Utils Supply Chain]. Detecting such threats requires moving beyond simple signature matching to behavioral analysis and dynamic execution monitoring. Tools employing sandboxed execution and fuzz testing have demonstrated the ability to detect malicious behaviors in NPM and PyPI packages with high precision, significantly reducing false positives compared to static analysis alone [119]. Additionally, the concept of "software supply chain smells" has been introduced to identify structural indicators of risk, such as inactive maintainers or suspicious install scripts, which often precede active compromises [54].

The interplay between versioning strategies and these risks remains a subject of intense debate. While pinning dependencies is often advocated to prevent the immediate ingestion of malicious updates, it paradoxically increases the risk of retaining known vulnerabilities for extended periods, effectively freezing the application in a vulnerable state [70]. Counterfactual analyses suggest that for large dependency graphs, pinning can be futile and even harmful, as it prevents the automatic propagation of security fixes and increases the likelihood of dependency conflicts [70]. Conversely, floating version constraints facilitate the rapid adoption of patches but expand the attack surface for malicious package updates. Recent findings suggest that a hybrid approach, utilizing floating constraints alongside lockfiles, may offer a more balanced defense, though this does not fully resolve the issue of silent failures or deep transitive risks [240]. Ultimately, addressing cascading, transitive, and silent risks requires a shift from reactive

scanning to proactive, context-aware governance that integrates graph-theoretic analysis, behavioral monitoring, and enhanced visibility into the entire dependency lifecycle.

Social Engineering and Build Pipeline Poisoning

The security landscape of the modern software supply chain has shifted dramatically from purely technical vulnerabilities to sophisticated campaigns that exploit human trust and automated workflows. A primary vector for these compromises involves social engineering attacks targeting open-source software (OSS) development teams, where adversaries manipulate team dynamics to gain commit access and inject malicious code. Research indicates that attackers specifically exploit the collaborative nature of OSS, leveraging interactions among team members to bypass traditional security perimeters [Leveraging Team Dynamics to Predict Open-source Software Projects' Susceptibility to Social Engineering Attacks]. This approach was starkly illustrated in the XZ Utils supply chain attack, where adversaries did not simply inject code but manipulated the entire software engineering process, including community management and continuous integration/continuous deployment (CI/CD) configurations, to establish long-term legitimacy and control [112]. Such attacks demonstrate that the threat model has expanded to include the weaponization of software engineering practices themselves, rendering source-level analysis insufficient when the build environment or the maintainers are compromised [255].

Once an attacker gains a foothold through social engineering, the focus often shifts to poisoning the build pipeline, a critical stage where source code is transformed into deployable artifacts. The build process represents a unique attack surface due to the complexity of multi-component systems and the prevalence of build non-determinism, which can mask malicious modifications [167]. Empirical studies reveal that a significant portion of supply chain attacks, approximately 23.8%, specifically exploit build vulnerabilities, with build script injection and dependency confusion being among the most prevalent vectors [167]. The danger lies in the fact that build code is functionally equivalent to program code yet is often excluded from standard security checks; poisoned build systems can alter final binaries without modifying the source code, effectively circumventing tools designed to detect program code vulnerabilities [106]. This capability allows attackers to disable security checks within the pipeline itself, creating a scenario where the output is untrustworthy despite the source appearing benign.

The vulnerability of CI/CD pipelines is exacerbated by the widespread adoption of automated workflows on platforms like GitHub, which introduce new attack surfaces through script runtimes and configuration files. Large-scale measurements indicate that script runtimes are prone to code hiding, and updates to scripts are often not performed in a timely manner, providing attackers with opportunities to embed malicious logic [100]. Furthermore, the reliance on a few core scripts creates single points of failure, and even scripts from verified creators may contain severe vulnerabilities that can be exploited to disrupt large software ecosystems [100]. Automated security assessments of GitHub Actions workflows have identified tens of thousands of security issues, highlighting that the problem is pervasive and demands immediate attention [84]. The lack of transparency in build environments further complicates defense; for instance, a vast majority of commonly used artifacts are not built using transparent CI/CD pipelines, forcing users to place blind trust in the build environment rather than verifying binary-source equivalence [150].

Defending against these sophisticated attacks requires a shift from purely code-centric analysis to monitoring team dynamics and ensuring build environment integrity. Traditional methods that focus solely on dependency verification ignore the build system itself, leaving a critical gap in security posture [106]. Novel strategies propose modeling the information and behavior permissions of build automation as if it were program code, a concept known as development phase isolation, which has shown promise in detecting poisoned test files involved in attacks like XZ Utils [106]. Additionally, leveraging signatures and patterns of team interactions offers a programming language-agnostic approach to predicting susceptibility to social engineering, assessing the artifacts of team behavior rather than the code itself [Leveraging Team Dynamics to Predict Open-source Software Projects' Susceptibility to Social Engineering Attacks]. However, current mitigation efforts face challenges; for example, while reproducible builds offer a robust solution to verify artifact integrity, variations in build environments and complex dependency graphs often lead to build failures, hindering widespread adoption [150].

The intersection of social engineering and pipeline poisoning creates a compound threat where human manipulation facilitates technical compromise. Attackers utilize social engineering to trick developers into approving malicious pull requests or altering CI/CD configurations, effectively bypassing automated safeguards [255]. This synergy between human error and automated vulnerability is evident in the finding that malicious actors often leverage seemingly beneficial contributions to project infrastructure to gain trust before executing an attack [112]. Consequently, effective

defense mechanisms must address both the human element and the technical infrastructure. While tools exist to detect anomalous commits based on metadata and cohesion analysis, the rarity and stealth of these attacks make automated detection challenging [125] [154]. The industry is increasingly recognizing that securing the supply chain requires a holistic view that integrates social dynamics, build process isolation, and rigorous verification of CI/CD workflows to prevent the injection of malicious code that traditional source analysis cannot see.

Structural Analysis and Transparency Mechanisms

Ensuring the integrity of the software supply chain necessitates robust metadata standards and verification mechanisms, with the Software Bill of Materials (SBOM) emerging as a foundational tool for transparency. However, the efficacy of SBOMs is currently compromised by significant inaccuracies in component identification, hidden dependencies, and inconsistent tooling, which collectively foster a false sense of security. To address these deficiencies, the research community is increasingly advocating for a transition from static lists to graph-based representations and verifiable artifact resolution systems that can capture the dynamic and complex nature of modern software dependencies.

The fundamental premise of SBOMs is to provide a machine-readable inventory of software components, yet empirical evidence suggests that current generation tools frequently fail to produce complete or accurate representations. Studies analyzing the Python ecosystem reveal systematic issues related to dependency versions, metadata files, and remote dependencies, often stemming from a lack of standardized metadata formats [194]. Similarly, investigations into Java projects demonstrate that popular SBOM generation tools exhibit poor inter-tool consistency, with package detection rates varying drastically between 7.84% and 12.77% across different languages and tools [140]. This inconsistency is not merely a technical nuisance but a critical security flaw; when tools cannot agree on the components present in a system, vulnerability assessments become unreliable, leading to either missed threats or overwhelming false positives [120]. Furthermore, the accuracy of license information in generated SBOMs is often below 20%, severely hindering compliance efforts [140].

A primary driver of these inaccuracies is the prevalence of hidden dependencies and component variants that escape traditional manifest-based analysis. Research highlights two specific mismatch patterns: hidden code-level dependencies where application code directly utilizes transitive dependencies without explicit declaration, and component variants such as clones or shaded jars that scanners fail to identify consistently [35]. These blind spots are particularly dangerous because they allow vulnerabilities to propagate undetected; for instance, a vulnerability in a transitive dependency accessed via deep code paths may remain unreported if the SBOM only reflects declared manifest dependencies. Additionally, the phenomenon of "component variants" means that identical vulnerable code may exist under different identities, evading detection by tools that rely on standard package identifiers [35]. The inability of current tools to cover all Component Inclusion Mechanisms (CIMs) results in SBOMs with significant ambiguity, suggesting that a security-grade SBOM is currently unachievable with existing technology [184].

To mitigate these structural weaknesses, there is a growing consensus on the need for graph-based representations and more rigorous artifact resolution. Approaches like VGraph propose consolidating SBOM and Software Composition Analysis (SCA) data into a knowledge graph, enabling the identification of concentrated risk points and revealing that vulnerabilities predominantly emerge at deeper dependency levels [39]. By modeling vulnerability relationships over dependency structures rather than treating scanner outputs as independent records, graph-theoretic methods can predict cascading vulnerability chains with higher accuracy [20]. Complementing this, systems like OmniBOR introduce a minimalistic scheme for creating artifact dependency graphs that track every software artifact incorporated into a build, embedding unique identifiers to ensure verifiable resolution across the supply chain [34]. This shift toward graph-based models allows for a more holistic view of how vulnerabilities propagate through complex, transitive dependencies, addressing the limitations of flat SBOM lists.

Beyond structural representation, the integrity of the SBOM generation and distribution process itself requires enhancement through cryptographic attestation and kernel-level monitoring. Traditional methods relying on static analysis or build manifests are susceptible to manipulation and often miss dynamically loaded dependencies. Frameworks like Bomfather leverage eBPF for kernel-level monitoring to capture build dependencies in real-time, constructing Merkle trees based on observed file content to provide tamper-evident identifiers [117]. Similarly, the Automatic Bill of Materials (ABOM) technique embeds dependency metadata directly into binaries at compile time using compressed Bloom filters, offering a zero-touch solution for dependency identification that does not rely on developer enumeration [71]. These approaches aim to root trust in the actual build process and binary artifacts rather than potentially flawed or manipulated metadata files.

Despite these advancements, challenges regarding the sharing and privacy of SBOM data persist. While transparency is vital, organizations often hesitate to share full SBOMs due to intellectual property concerns or the risk of exposing

vulnerability information to attackers. Solutions like zkSBOM utilize zero-knowledge sets to allow consumers to query for vulnerabilities without revealing the full component list, balancing transparency with confidentiality [272]. Furthermore, the extension of SBOM concepts to AI systems, termed AIBOM or TAIBOM, introduces new complexities regarding dynamic data-driven components and model provenance, requiring structured dependency models that traditional SBOMs cannot accommodate [64] [37]. The evolution from static inventories to dynamic, graph-based, and cryptographically verifiable systems represents a critical trajectory for securing the software supply chain against increasingly sophisticated threats.

Limitations of Current SBOM and SCA Tooling

The foundational premise of Software Composition Analysis (SCA) and Software Bills of Materials (SBOMs) rests on the assumption that declared dependencies in manifest files accurately reflect the code executed at runtime. However, this assumption is frequently violated, creating a critical disconnect between the inventory of components and the actual attack surface of an application. The reliance on static metadata leads to the prevalence of "hidden dependencies," where application code directly invokes functionality from transitive dependencies without explicit declaration in the project's build configuration. This phenomenon, often resulting from deep access patterns in ecosystems like Java, allows vulnerabilities to propagate undetected by standard scanners that only parse manifest files [35]. Consequently, SCA tools generate significant false negatives, failing to report vulnerabilities that are actively reachable within the compiled artifact because the dependency path exists only at the code level, not the component level.

Compounding the issue of hidden dependencies is the challenge of component variants, which include cloned, shaded, or repackaged libraries. Standard SCA tools typically identify components based on package names and version numbers found in manifest files, rendering them ineffective against modified binaries that retain vulnerable code but lack standard identifiers [35]. Research demonstrates that bytecode-centric approaches, such as Jaralyzer, possess superior capability in detecting these variants by analyzing the actual binary content rather than relying on metadata, yet such tools are not yet universally adopted in mainstream SCA pipelines [222]. The inability to distinguish between declared but unused dependencies and those actually invoked leads to an overwhelming volume of false positives. Empirical studies indicate that without reachability analysis, downstream vulnerability scanners can produce false positive rates as high as 97.5%, flagging vulnerabilities in code paths that are never executed [127]. This noise contributes to alert fatigue, causing development teams to deprioritize critical security fixes. Advanced function call analysis has shown promise in pruning these false alarms by up to 63.3%, yet these methods remain largely experimental or siloed from standard SBOM consumption workflows [127].

The fragmentation of the SBOM ecosystem further exacerbates these technical limitations. The coexistence of competing standards, primarily SPDX and CycloneDX, has led to a disjointed tool landscape where interoperability is severely hindered [213]. Empirical analyses reveal that different SBOM generation tools produce inconsistent outputs for the same codebase, with inter-tool consistency rates for package detection dropping as low as 7.84% across different languages [140]. This lack of standardization prevents organizations from reliably exchanging SBOM data across their supply chains or integrating diverse security tools into a unified workflow. Furthermore, the adherence gap between standards and tools is persistent; many tools fail to satisfy key government-mandated requirements, such as the minimum elements defined by the NTIA, posing risks to policy compliance [140].

The handling of Vulnerability Exploitability eXchange (VEX) statements further illustrates the immaturity of current tooling. VEX is designed to allow vendors to communicate whether a specific vulnerability is exploitable in their product, thereby suppressing irrelevant alerts. However, inconsistencies in how different tools parse and apply VEX statements can lead to contradictory security postures. If a tool fails to correctly map a VEX statement to a hidden dependency path, a vulnerability declared as "not affected" may remain unmitigated if the exploit path exists outside the scope defined by the manifest [35]. Practitioners have reported significant discrepancies in VEX handling between image scanning and SBOM-based scanning, with some tools ignoring VEX documents entirely or failing to manage attestation correctly [35]. The complexity of interpreting VEX is also a barrier; there is a noted shortage of professionals capable of correctly analyzing these documents, and automated generation tools often produce low-quality or inconsistent VEX data [35].

Moreover, the static nature of traditional SBOMs fails to capture the dynamic realities of modern software execution, such as environment drift or runtime behavior. Conventional SBOMs provide a snapshot of ingredients but cannot verify if a component is actually loaded or executed under specific operational conditions [118]. This limitation is critical because a vulnerability in a library that is never loaded at runtime poses no immediate threat, yet static analysis treats it with the same severity as an actively exploited flaw. Approaches utilizing kernel-level monitoring or function-call reachability analysis have shown promise in addressing these gaps by capturing build-time and runtime dependencies more accurately, yet these methods remain largely experimental or siloed from standard SBOM consumption workflows [117]. The cumulative effect of these limitations—hidden dependencies, component variants, format fragmentation, and inconsistent VEX handling—is a security posture that is often either dangerously optimistic

due to missed vulnerabilities or paralyzingly pessimistic due to unactionable noise. Without a shift toward runtime-aware analysis and stricter adherence to interoperable standards, the promise of SBOMs as a foundational element of supply chain security remains unrealized.

Graph-Based Approaches for Dependency Visualization

To overcome the inherent limitations of linear Software Bill of Materials (SBOMs), which often fail to capture the intricate transitive relationships within modern software ecosystems, researchers are increasingly adopting graph-theoretic methodologies. These approaches integrate dependency and vulnerability data into holistic knowledge graphs, enabling the identification of concentrated risk points and the prediction of multi-vulnerability attack chains. By modeling software projects as heterogeneous graphs, it becomes possible to detect cascading failures and provide a more accurate assessment of real-world risk than traditional per-CVE scanning, particularly in ecosystems characterized by complex transitive dependencies. A foundational contribution to this domain is VGraph, which consolidates SBOM and Software Composition Analysis (SCA) outputs into a unified graph representation. This methodology allows for the formal analysis of theoretical properties and the management of conflicts between disparate data sources. Empirical evaluation of VGraph on Java projects revealed that vulnerabilities predominantly emerge at a dependency depth of three or higher, indicating that direct dependencies are often more secure than their transitive counterparts, and highlighting the existence of concentrated risk points reachable through numerous paths [39].

Building upon the structural insights provided by static graph models, recent research has introduced machine learning techniques to predict multi-vulnerability attack chains. Traditional SBOM-based pipelines typically treat scanner findings as independent records, ignoring the cascaded interactions that characterize sophisticated supply chain compromises. To address this, novel approaches represent vulnerability-enriched CycloneDX SBOMs as heterogeneous graphs where nodes capture software components and known vulnerabilities connected by typed relations. By training Heterogeneous Graph Attention Networks (HGAT) on these structures, researchers have achieved high accuracy in predicting component vulnerability associations. Furthermore, framing the discovery of cascading vulnerabilities as a link prediction problem using Multi-Layer Perceptrons has demonstrated significant efficacy in identifying documented multi-vulnerability chains, thereby shifting the paradigm from isolated CVE analysis to systemic risk forecasting [36] [20].

The necessity for graph-based analysis is further underscored by the limitations of coarse-grained, package-level assessments which often produce false positives or fail to accomplish whole-ecosystem propagation analysis. To quantify the dynamic impact of vulnerability propagation, frameworks such as the Vulnerability Propagation Scoring System (VPSS) have been proposed. These systems utilize hierarchical worklist-based algorithms to perform call-graph-level analysis, offering a dynamic metric to measure the scope and evolution of vulnerability impacts across entire ecosystems like Java Maven [113]. Similarly, in the NPM ecosystem, where dependency resolution rules are complex and often misinterpreted by simple reachability analysis, knowledge graph-based dependency resolution has been employed to construct complete dependency-vulnerability graphs. This approach, exemplified by the DTRResolver algorithm, precisely resolves dependency trees and transitive propagation paths, revealing that existing tools often underestimate the scale of vulnerability exposure due to incorrect dependency resolution [122].

Beyond mere detection, graph-based models are being leveraged to enhance the precision of vulnerability management by incorporating semantic context and execution paths. Tools like DEPTEx treat supply chain risk as an emergent property rather than an intrinsic component attribute. By fusing Code Property Graph (CPG) slicing with Large Language Model semantic verification, these platforms calculate a vulnerability's true operational blast radius through Execution Path Dominance, significantly reducing alert fatigue caused by unreachable vulnerabilities [86]. This aligns with findings that a vast majority of flagged vulnerabilities in standard SBOM scans reside in unreachable code; function call analysis integrated into graph structures can prune a significant percentage of these false alarms, validating a two-stage approach of accurate SBOM generation followed by reachability enrichment [127].

The application of graph theory also extends to logical attack graphs for modeling threat propagation in software supply chains. Extensions to tools like MulVal now incorporate predicates for supply chain assets, such as build environments and software artifacts, enabling the reasoning of threat propagation from upstream compromises to downstream victims. This formalization allows for the uncovering of non-trivial attack scenarios, such as the sequential compromises seen in the 3CX and SolarWinds incidents, which linear models fail to capture [124]. Furthermore, the structural characteristics of specific ecosystems, such as the Large Language Model supply chain, exhibit a "locally dense, globally sparse" topology where critical vulnerabilities propagate rapidly through high-degree hub nodes. Understanding these topological features is essential for developing targeted mitigation strategies against cascading risks [21].

Despite the advancements, challenges remain regarding the accuracy of the underlying data feeding these graph models. Inconsistencies in SBOM generation and the presence of hidden code-level dependencies that are not represented in manifest-derived SBOMs can lead to incomplete graph structures and inconsistent vulnerability reporting. Studies indicate that hidden dependencies and component variants (clones) create blind spots in current SCA tools, necessitating richer dependency representations that go beyond package-level metadata to include code-level relationships [35] [51]. Consequently, the efficacy of graph-based approaches is contingent upon the fidelity of the input data, driving the need for improved SBOM generation tools and standardized formats that accurately reflect the true dependency landscape.

Extending Transparency to AI and Binary Artifacts

Traditional Software Bills of Materials (SBOMs) have long served as the cornerstone for software supply chain transparency, providing inventories of components and dependencies. However, the rapid integration of artificial intelligence and the complexities of modern binary distribution have exposed significant limitations in these static formats. Conventional SBOMs struggle to capture the dynamic, data-driven nature of AI systems, specifically failing to track data lineage, model provenance, and the iterative processes inherent in machine learning workflows [64]. This insufficiency has prompted the research community to develop specialized extensions, most notably the AI Bill of Materials (AIBOM) and the Trusted AI Bill of Materials (TAIBOM), which aim to broaden the remit of supply chain documentation to encompass AI-specific artifacts such as training datasets, model weights, and inference pipelines [37]. Unlike traditional software, AI systems evolve continuously through retraining and data drift, necessitating frameworks that support traceable, evolving records rather than static inventories [64]. The TAIBOM framework, for instance, introduces a structured dependency model that conceptualizes relationships between data, code, and AI system objects, while integrating cryptographic attestations to verify component provenance across heterogeneous environments [64]. Furthermore, the development of the AIBOM specification through global, multi-stakeholder action research highlights the community effort to standardize these AI-specific extensions within existing frameworks like SPDX [65].

Parallel to the challenges in the AI domain, a critical gap exists between declared software composition and the actual contents of binary artifacts. Standard SBOM generation tools often rely on metadata files or manifest analysis, which can lead to inaccuracies regarding hidden dependencies, component variants, and dynamically loaded libraries [35]. Research indicates that current tools frequently miss code-level dependencies that are not explicitly declared at the component level, creating blind spots in vulnerability management [35]. To address this discrepancy, researchers have proposed the Automatic Bill of Materials (ABOM), a technique that embeds dependency metadata directly into binaries at compile time. By storing hashes of input source code files within Compressed Bloom Filters inside the binary, ABOM provides a zero-touch, language-independent solution for verifying the actual composition of software artifacts, thereby mitigating the risks of supply chain attacks that exploit the divergence between declared and actual dependencies [71]. This approach contrasts with traditional methods that require developers to explicitly enumerate dependencies, a process prone to human error and manipulation [149]. Similarly, OmniBOR proposes a minimalistic scheme to create artifact dependency graphs by embedding identifiers into built software, enabling verifiable resolution across the supply chain [34]. Kernel-level monitoring frameworks like Bomfather further enhance this by using eBPF to capture build-time dependencies transparently, accounting for conditional compilation and dynamic loading that static analysis often misses [117].

The secure sharing and verification of these enhanced bills of materials introduce further complexities, particularly concerning the tension between transparency and confidentiality. Vendors often hesitate to disclose full SBOM details due to intellectual property concerns or the fear that revealing specific vulnerabilities could aid attackers [173]. To resolve this, blockchain-enabled architectures and verifiable credentials have been proposed to facilitate secure SBOM sharing with selective disclosure. These mechanisms allow consumers to query redacted SBOMs for specific security answers without accessing the underlying confidential metadata, ensuring integrity through tamper-evident representations [173]. Furthermore, blockchain technologies offer a decentralized approach to validate the integrity of dependencies, addressing the susceptibility of SBOM generation processes to stealthy manipulation by malicious insiders [149]. Complementary approaches include privacy-preserving sharing mechanisms like zkSBOM, which utilizes zero-knowledge sets to cryptographically commit to SBOM components, allowing users to verify vulnerability status without revealing the full list of ingredients [272]. Another system, Petra, empowers vendors to distribute redacted SBOM data using selective encryption, enabling efficient and confidentiality-preserving integrity proofs [173].

Despite these advancements, the ecosystem faces persistent challenges regarding tool adherence to standards and the consistency of generated data. Empirical analyses reveal significant gaps between SBOM standards and the tools designed to implement them, with issues ranging from inadequate policy compliance to poor inter-tool consistency and mediocre accuracy in package detection [140]. The lack of a shared understanding of what constitutes a "component" further exacerbates these issues, leading to ambiguity and blind spots that undermine the security grade of current SBOM implementations [184]. Additionally, the rise of AI-generated code and "vibe coding" introduces new vectors for supply chain compromise, such as source map exposure and packaging configuration drift, which traditional static

analysis tools often fail to detect [24]. Consequently, the evolution toward AIBOMs, ABOMs, and blockchain-secured sharing represents a necessary shift from reactive verification to proactive, structurally embedded transparency capable of addressing the nuanced threats of modern software and AI supply chains.

Conclusion

The literature reveals a software supply chain landscape that has fundamentally shifted from a linear development pipeline into a complex, interdependent ecosystem where traditional security perimeters have dissolved. The ubiquity of open-source components, now comprising the vast majority of commercial codebases, has expanded the attack surface to include not only direct dependencies but also deep transitive chains, build infrastructures, and increasingly, the AI models and datasets that underpin modern applications. This evolution has rendered conventional static analysis and rule-based scanners insufficient, as they struggle with the semantic nuances of AI-generated code, opaque industrial binaries, and sophisticated social engineering campaigns that weaponize human trust to compromise build pipelines. The emergence of the Large Language Model supply chain introduces unique topological vulnerabilities, where "locally dense, globally sparse" networks create single points of failure capable of cascading risks to thousands of downstream projects, often remaining undetected for years due to the black-box nature of model artifacts and configuration files [21].

In response to these escalating threats, the research community is driving a paradigm shift toward AI-enhanced and agentic defense mechanisms. There is a growing consensus that effective security requires moving beyond simple provenance tracking and static inventories toward dynamic, semantic-aware systems capable of autonomous reasoning and mitigation. Graph-based architectures and neuro-symbolic frameworks are proving essential for bridging the gap between abstract code structures and concrete vulnerability semantics, significantly reducing false positives by distinguishing between reachable and unreachable flaws. Furthermore, the integration of Large Language Models within multi-agent systems offers a promising avenue for proactive defense, enabling coordinated analysis across heterogeneous artifacts and the automated generation of explainable, context-aware remediation strategies. These agentic systems, often augmented by reinforcement learning, aim to transform the supply chain from a passive target into a self-defending entity capable of adapting to evolving attack vectors in real time [4].

However, the realization of this secure future is contingent upon resolving critical challenges related to data quality, standardization, and the reliability of automated tools. Current efforts are hampered by fragmented Software Bill of Materials standards, inconsistent tooling that fails to capture hidden dependencies, and a lack of high-fidelity, time-wise datasets that lead to poor generalization in machine learning models. The tension between transparency and confidentiality, particularly regarding the sharing of vulnerability data, necessitates advanced cryptographic solutions such as zero-knowledge proofs and blockchain-enabled attestation to ensure trust without exposing intellectual property. Moreover, the ethical implications of AI-driven security, including the risk of untrustworthy predictions and algorithmic bias, underscore the indispensable role of human oversight. Ultimately, securing the evolving software supply chain demands a holistic, defense-in-depth strategy that synthesizes rigorous governance frameworks, domain-specific adaptations for critical infrastructure, and a collaborative ecosystem where human expertise guides the deployment of autonomous, intelligent defense systems.